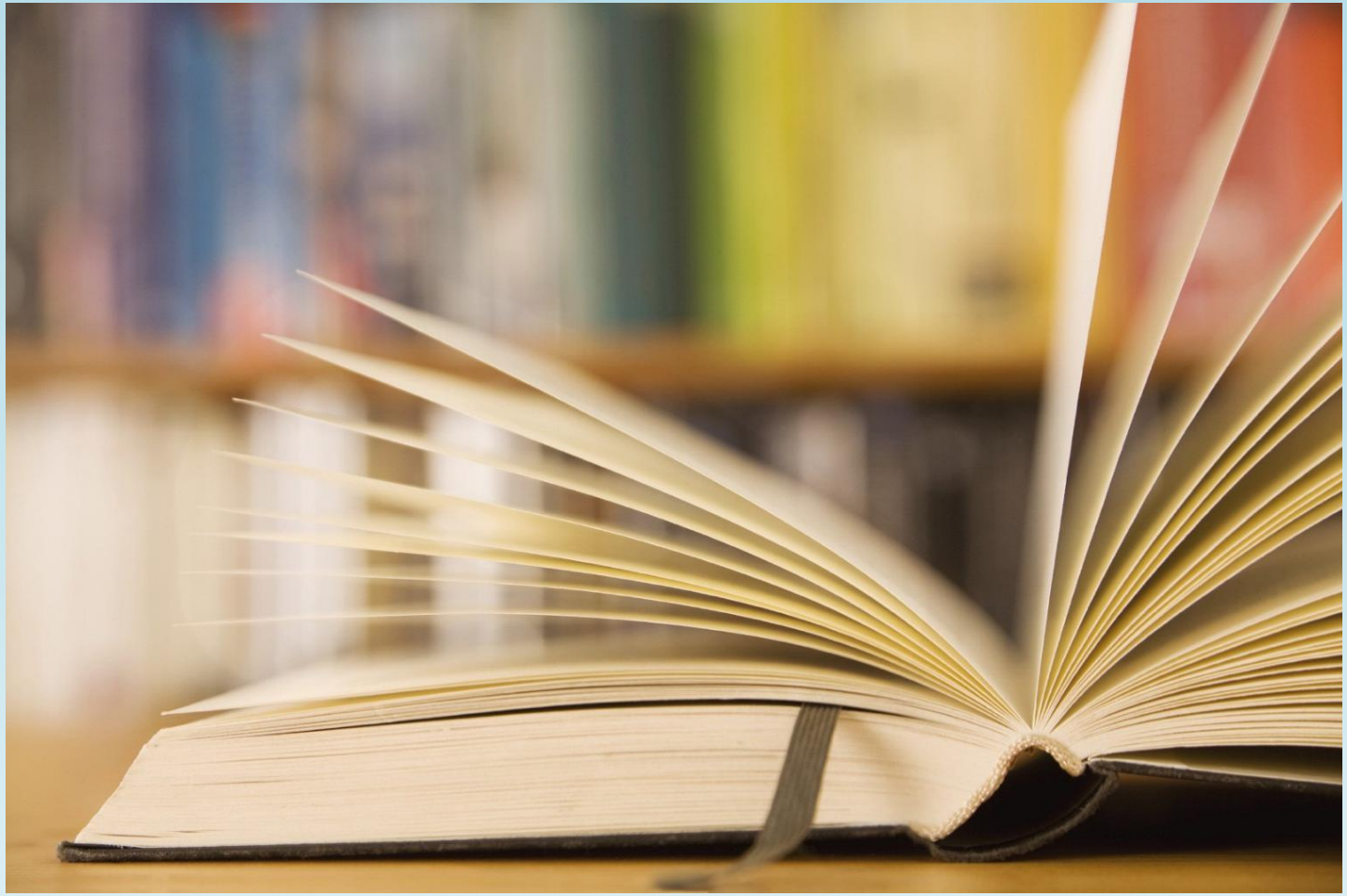


Project Methodology

BIS601

Project Management

- Project portfolio management: prioritise, select, manage
- Project management: choose system development methodology, create workplan, staffing, mechanisms to coordinate, monitor and control

Project Selection

- Size: What is the size? How many people needed?
- Cost: How much will project cost organisation?
- Purpose: Improve technical infrastructure? Support current business strategy? Improve operation? Demonstrate new innovation?
- Length: How long before completion? How long before value delivered to business?

Project Selection (cont.)

- Risk: How likely project succeed or fail?
- Scope: How much of organisation affected by the system?
- Economic value: RoI

Project Methodology Options

Project characteristics that will affect methodology selection decision:

- Clarity of user requirements: How well users and analysts understand functions and capabilities needed from new system?
- Familiarity with technology: How much experience the project team have with the technology that will be used?

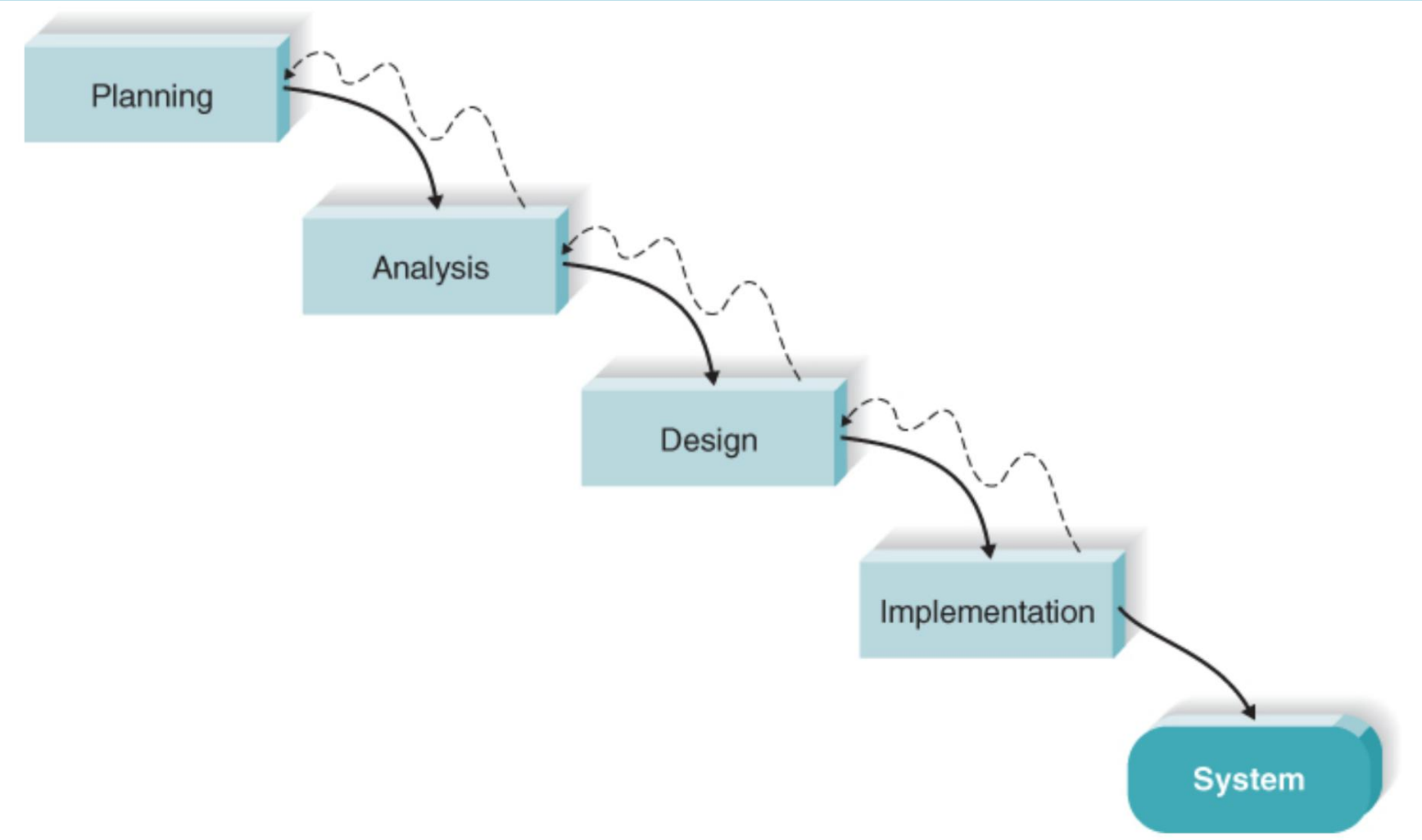
Project Methodology Options (cont.)

- System complexity: New system includes wide array of features? System has to integrate with many existing systems? Span multiple organisational units or even multiple organisations?
- System reliability: Highly reliable or some downtime tolerable?
- Short time schedule: Project time frame tight?
- Schedule visibility: Project sponsors, users, organisational managers anxious to see progress?

Project Methodology Options (cont.)

1. Waterfall Development
2. Rapid Application Development
3. Agile Development

1. Waterfall Development



Waterfall Development (cont.)

- Sequential, deliverables for approval as project moves from phase to phase
- Advantages:
  - Requirements identified long before programming begins
  - Requirements changes limited
- Disadvantages:
  - Long time elapses between completion of system proposal and delivery of system

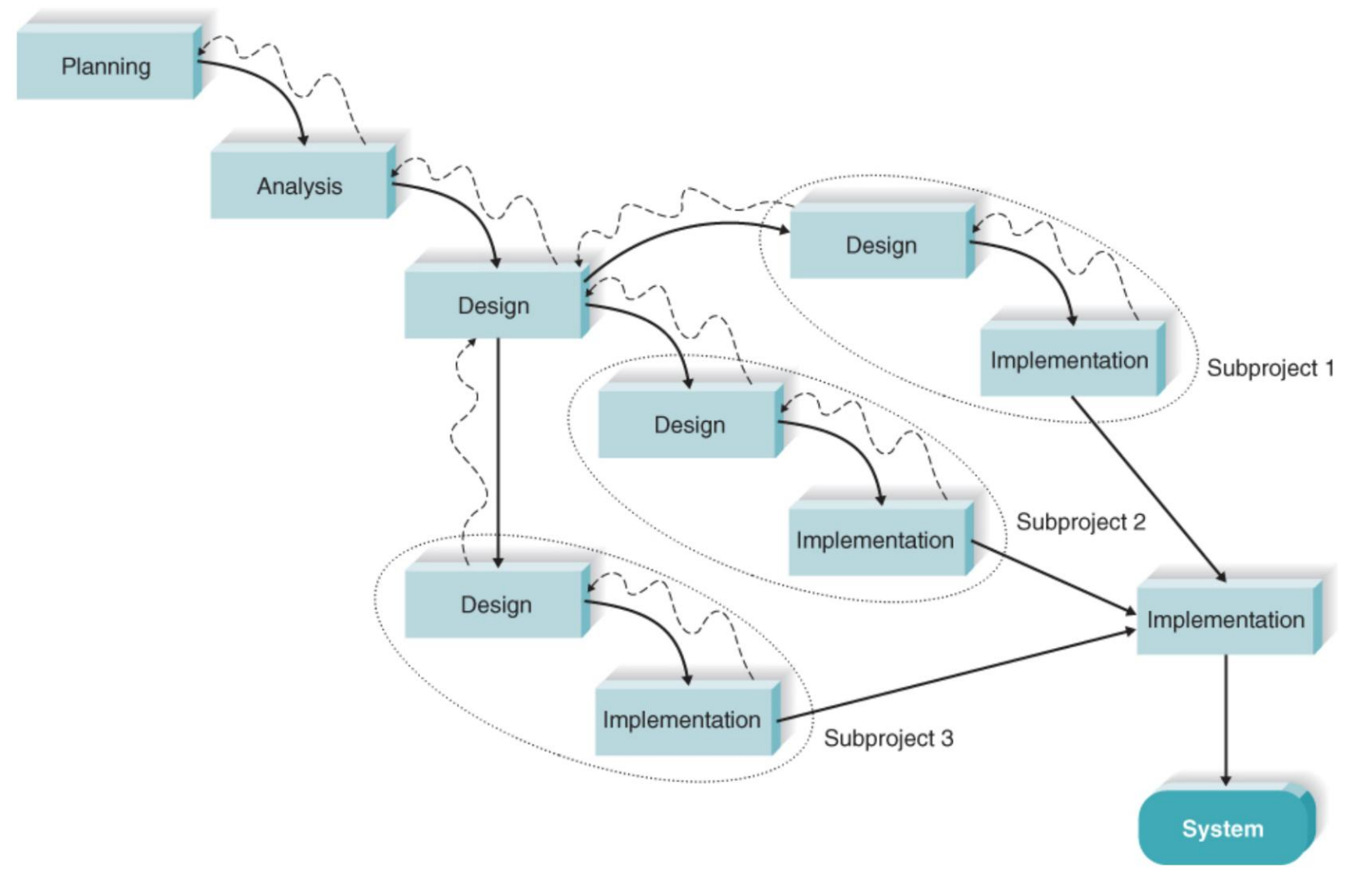
Waterfall Development (cont.)

- Deliverables are often a poor communication mechanism, so important requirements may be overlooked in volumes of documentation
- In dynamic business environment, may need considerable rework when implemented

Waterfall Development (cont.)

- Variants
  - Parallel development
    - General design for whole system
    - Project divided into series of subprojects that can be designed and implemented in parallel
    - Final integration

Waterfall Development (cont.)

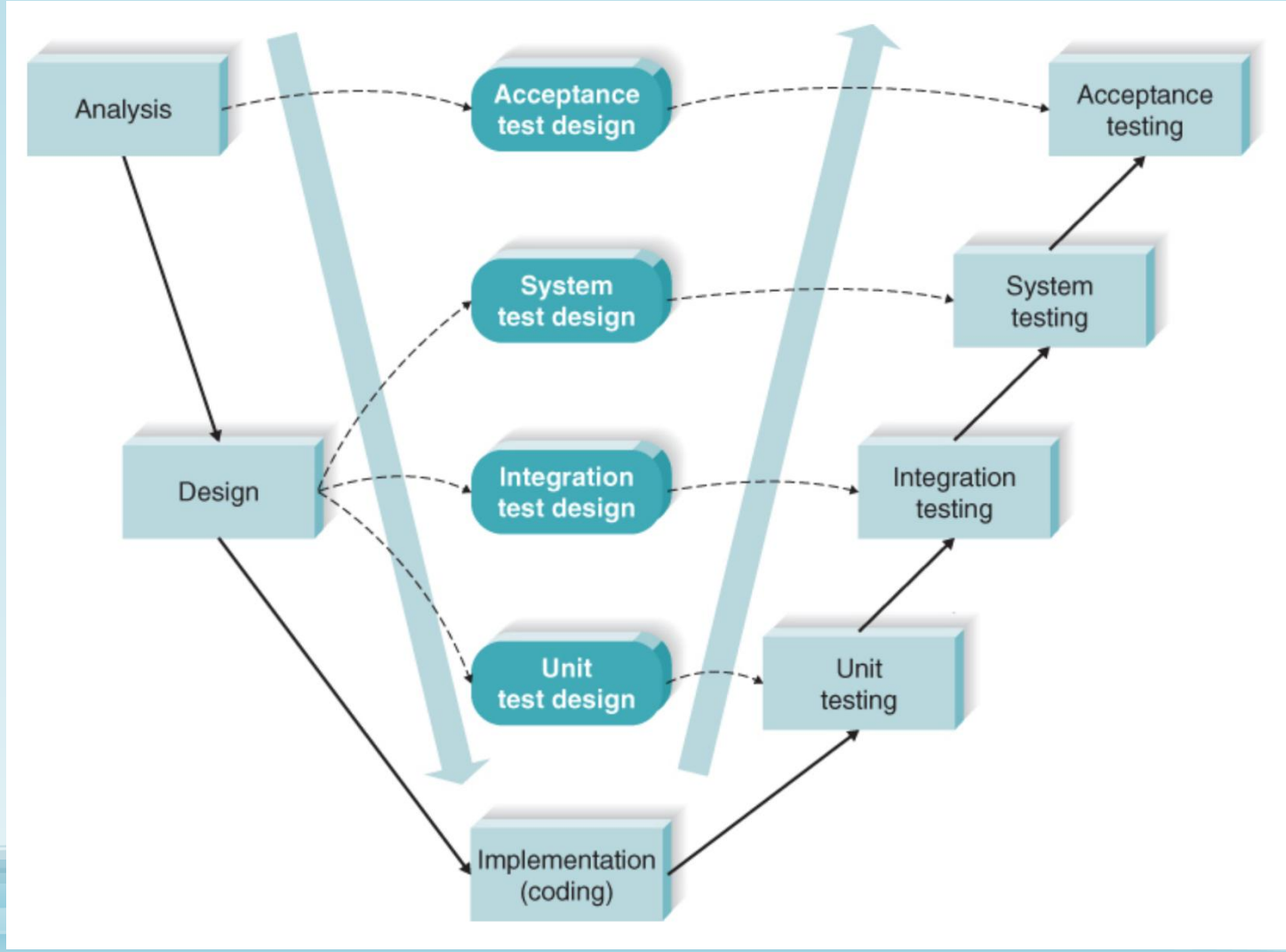


Waterfall Development (cont.)

- Reduce time required to deliver system, so changes in business environment less likely to produce the need for rework
- If subprojects not completely independent, design decisions in one project may affect another, integrating subprojects may be challenging

Waterfall Development (cont.)

- V-model



Waterfall Development (cont.)

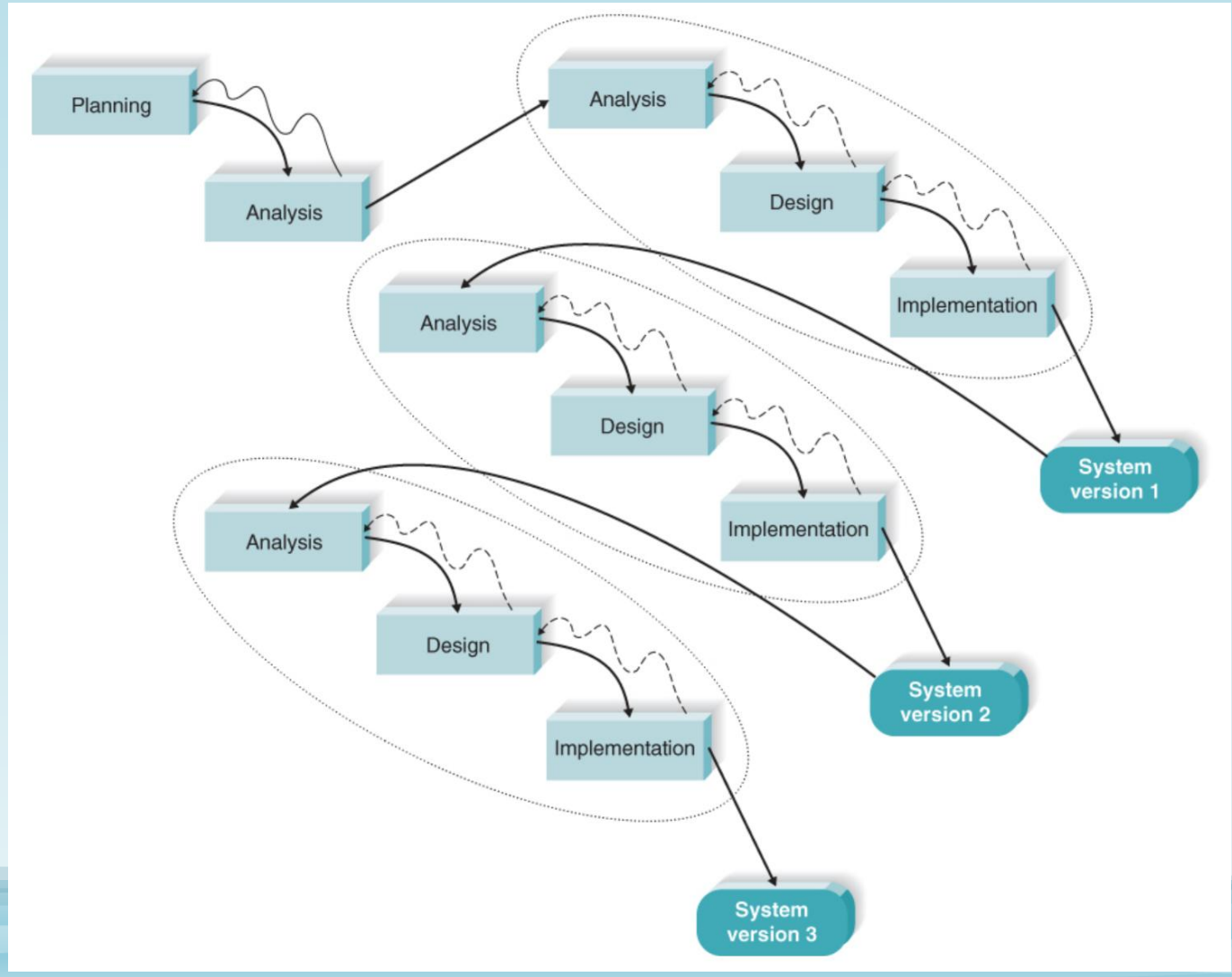
- More explicit attention to testing
- As requirements specified and components designed, testing for these components also defined
- Each level of testing clearly linked to part of analysis or design phase, helping to ensure high quality and relevant testing
- Not always appropriate for dynamic nature of business environment

2. Rapid Application Development

- Rapid application development (RAD): get some portion of system developed quickly and into hands of users for evaluation and feedback
- May introduce problem in managing user expectation: user expectations increase and system requirements expand during project (scope creep or feature creep)

Rapid Application Development (cont.)

- Iterative development

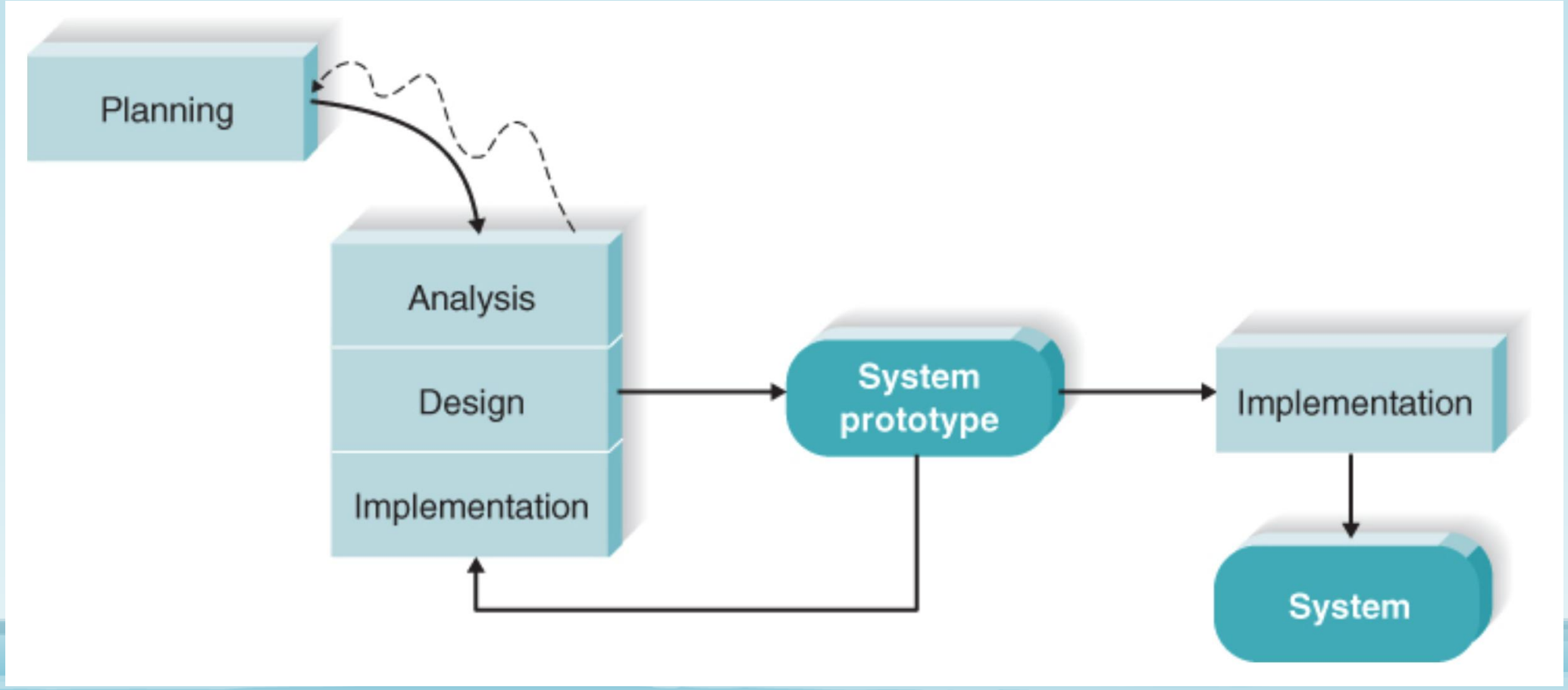


Rapid Application Development (cont.)

- Break overall project into series of versions
- Most important and fundamental requirements bundled into first version
- Feedback from users incorporated into next version
- Additional requirements identified and incorporated into subsequent versions
- Disadvantage: users begin to work with incomplete system

Rapid Application Development (cont.)

- System prototyping



Rapid Application Development (cont.)

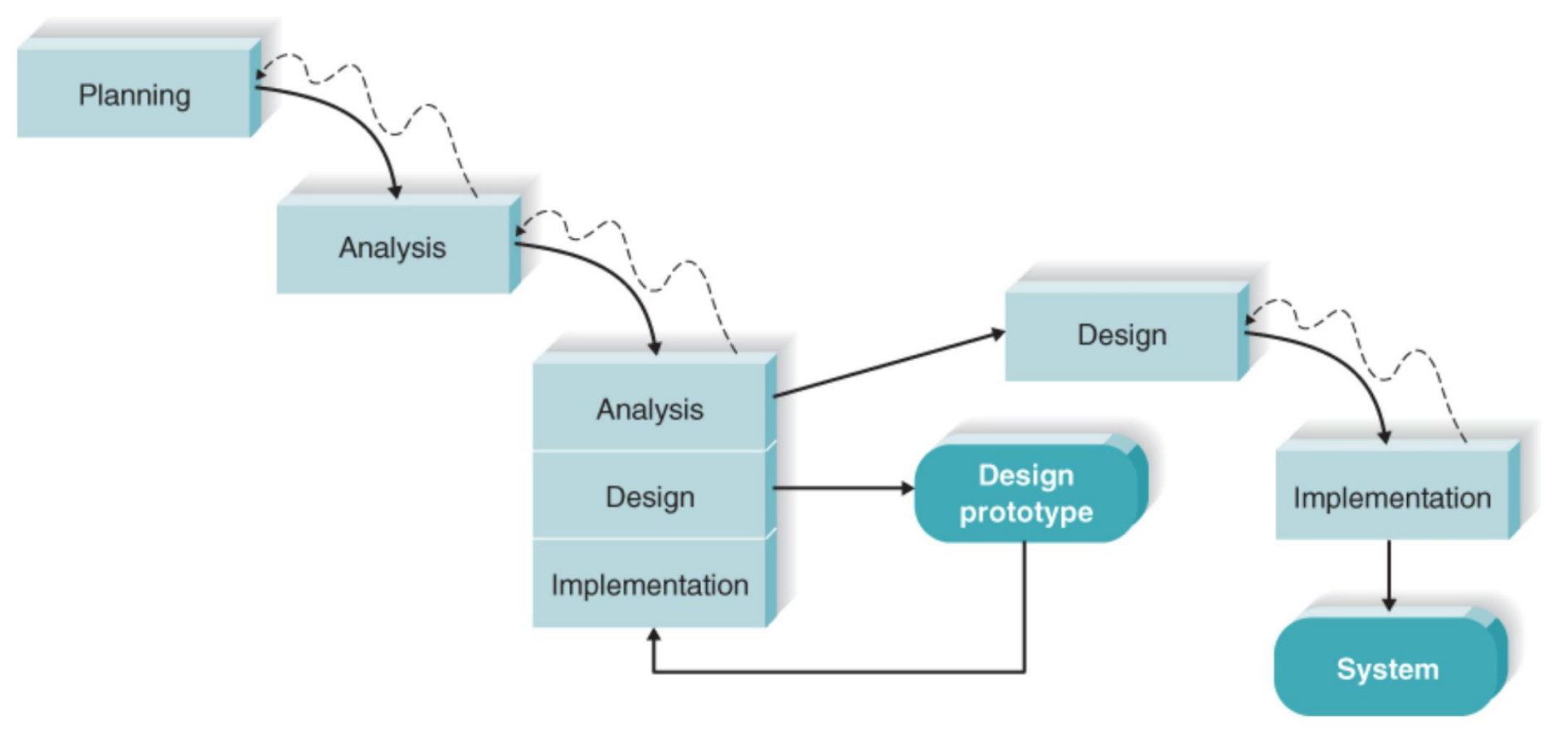
- Perform analysis, design, and implementation phases concurrently to quickly develop simplified version of proposed system and get evaluation and feedback from users
- Reanalyse, redesign, and reimplement second prototype
- Cycle continues until prototype provides enough functionality

Rapid Application Development (cont.)

- Advantages:
  - Reassure users that progress being made
  - Useful when users have difficulty in expressing requirements
- Disadvantages:
  - Lack of careful, methodical analysis before making design and implementation decisions

Rapid Application Development (cont.)

- Throwaway prototyping



Rapid Application Development (cont.)

- Use prototype primarily to explore design alternatives rather than as actual new system
- Design prototype: not to be working system, contain only enough details to enable users to understand issues under consideration
- Each of prototypes used to minimise risk associated with system
- Once issue resolved, project moves into design and implementation, design prototypes thrown away

Rapid Application Development (cont.)

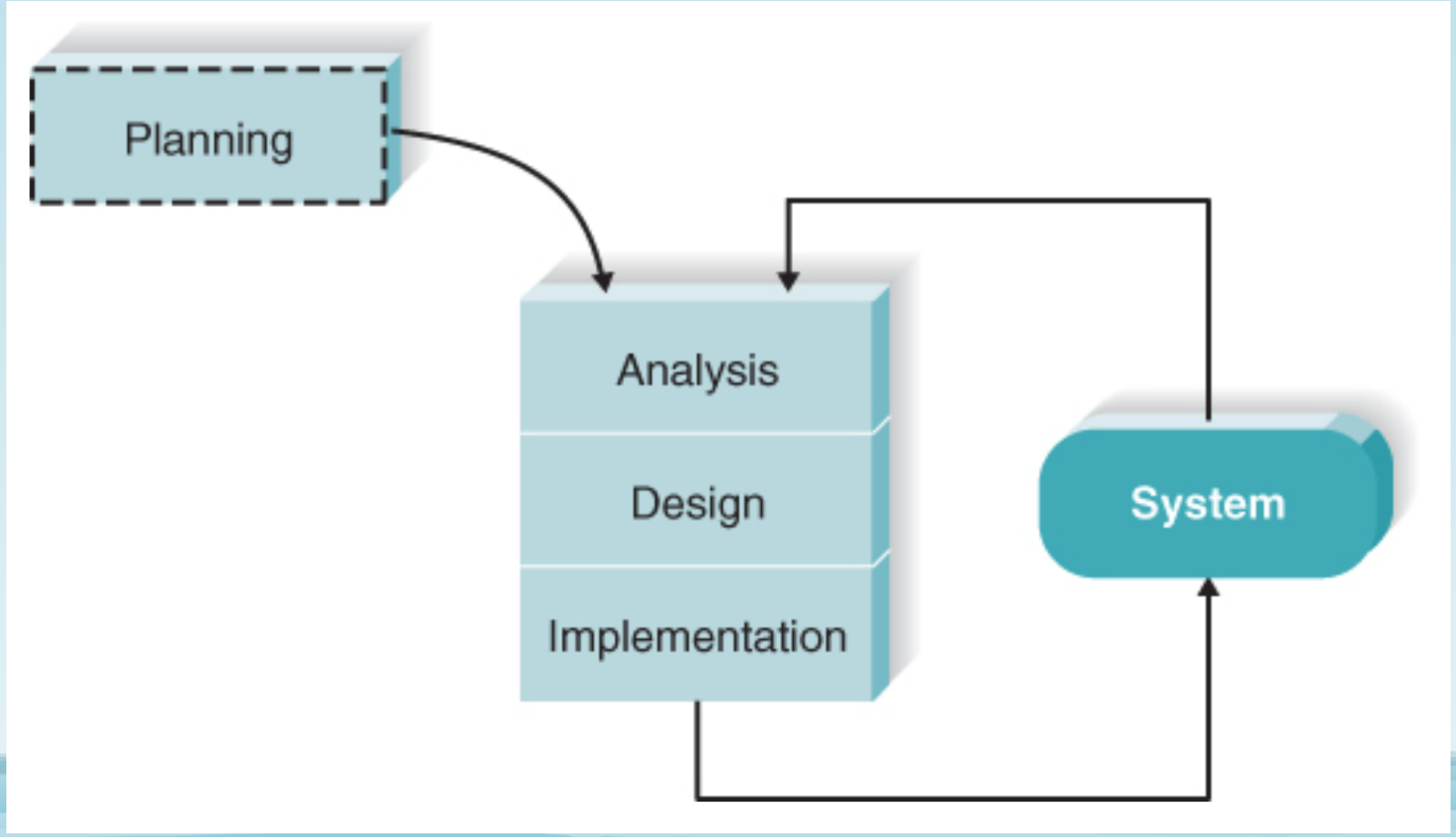
- System prototyping: prototypes evolve into final system
- Compared with system prototyping, throwaway prototyping may take longer to deliver final system, but produced more stable and reliable system

3. Agile Development

- Much of modelling and documentation overhead eliminated, prefer face-to-face communication
- Simple, iterative development; every iteration is a complete software project
- Cycles kept short (1-4 weeks)
- Focus on adapting to current business environment

Agile Development (cont.)

- Extreme programming (XP)

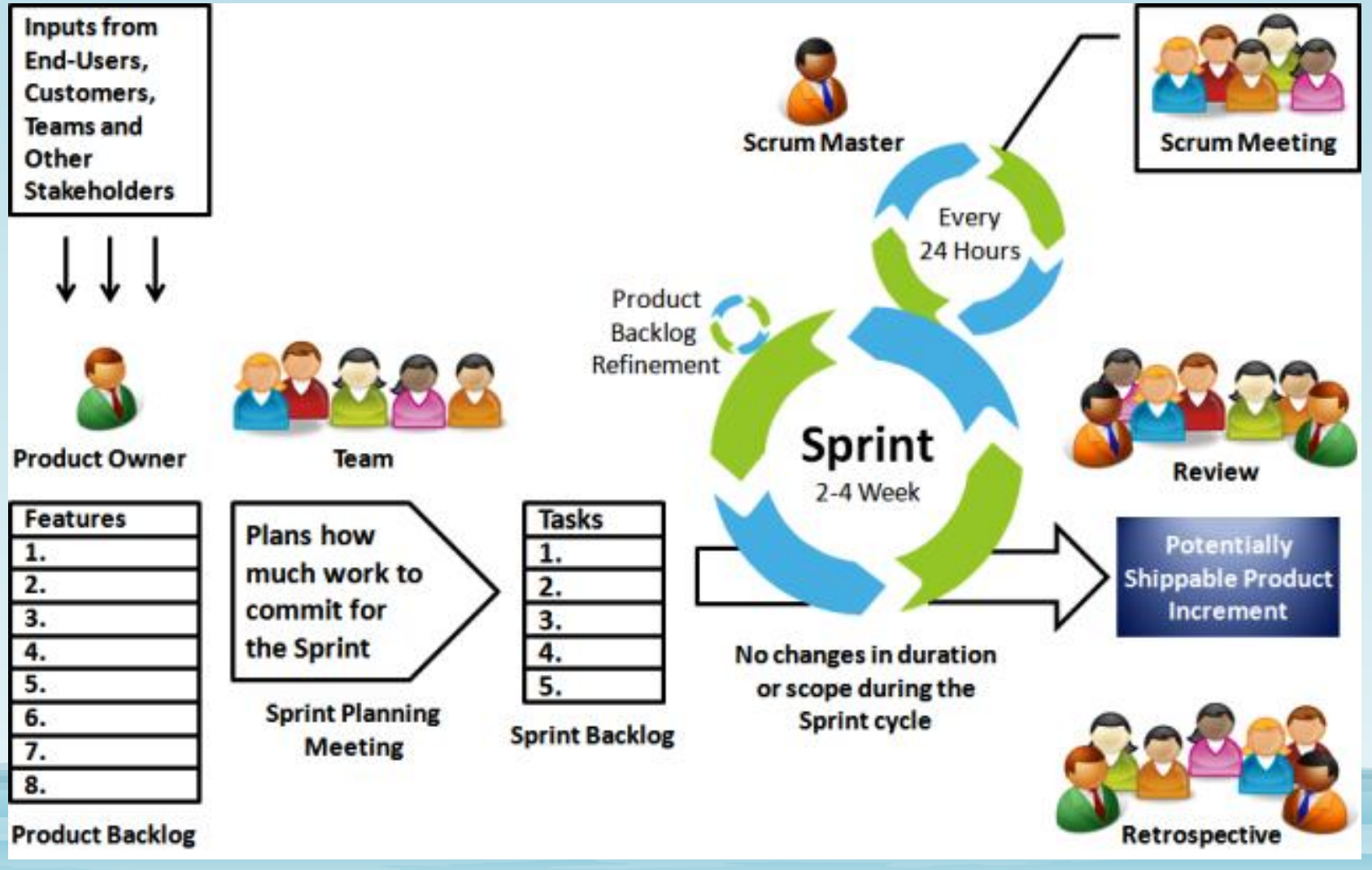


Agile Development (cont.)

- Begin with user stories describing what system needs to do
- Code in small, simple modules and test to meet those needs
- Users required to be available
- Highly motivated, cohesive, stable, and experienced teams
- Little analysis and design documentation → maintenance of large systems may be difficult

Agile Development (cont.)

- Scrum



Agile Development (cont.)

- Small teams, series of iterations (sprints)
- Obtain requirements → self-organised → form backlogs containing planned approach
- Scrum master guiding team's progress through backlogs with repeated sprints

Selecting a Methodology

Usefulness in Developing Systems	Waterfall	Parallel	V-Model	Iterative	System Prototyping	Throwaway Prototyping	Agile Development
With unclear user requirements	Poor	Poor	Poor	Good	Excellent	Excellent	Excellent
With unfamiliar technology	Poor	Poor	Poor	Good	Poor	Excellent	Poor
That are complex	Good	Good	Good	Good	Poor	Excellent	Poor
That are reliable	Good	Good	Excellent	Good	Poor	Excellent	Good
With short time schedule	Poor	Good	Poor	Excellent	Excellent	Good	Excellent
With schedule visibility	Poor	Poor	Poor	Excellent	Excellent	Good	Good